Lodz University of Technology

Final Report
Orient AI

Team Project
Tutors: Joanna Jojczyk & Zdzisława Rowińska

Aroa Aguiño Domínguez –> 905686@edu.p.lodz.pl
Maria Encina Díez del Río --> 905687@edu.p.lodz.pl
Mario Fernández Costello --> 905688@edu.p.lodz.pl
Naroa Genicio Iglesias --> 905689@edu.p.lodz.pl
David Sánchez Sánchez --> 905590@edu.p.lodz.pl
Jaime Esteban Reano --> 905805@edu.p.lodz.pl
Jorge Domínguez Palomo --> 905804@edu.p.lodz.pl

Index

Section 1: Research Context and Project Justification: Why Orient AI Needs to Exist

1.1 The Real-World Need: The Vocational Disorientation Crisis

This project did not begin in a laboratory or through a literature review, but rather through informal conversations with students at the Politechnika University of Łódź. Across different faculties, students repeatedly described the process of choosing a university degree as stressful, uncertain, and confusing. Many admitted that their decisions were often influenced by external pressures, a lack of guidance, or a process of elimination rather than a genuine understanding of their interests and strengths.

These observations suggested the existence of a broader problem: vocational disorientation. Rather than being an individual issue, it appeared to be a systemic weakness in current career guidance practices.

1.2 Validation of the Problem

To determine whether these concerns were still relevant, we conducted qualitative research involving both university and high-school students. Participants consistently expressed feelings of confusion and frustration when making educational decisions.

The feedback we received was unequivocal and highly revealing. Despite the passage of time and the increasing digitalization of education, the current generation reported experiencing the exact same frustrations we had faced.

Students highlighted several recurring issues:

- Lack of personalized guidance.
- Excessive amount of information without clear direction.
- Fear of making the wrong academic choice.
- Dissatisfaction with existing orientation methods.

The findings suggested that current career guidance systems are often unable to adapt to individual student needs and this is not a past issue, it is a current and ongoing problem.

1.3 Quantitative Validation Through Survey Research

To turn our ideas into a solid, data-driven machine learning project, we needed to back up the feedback with real data. So, a questionnaire was designed and distributed through Google Forms. The survey targeted high-school students who were actively considering their future educational paths. Through collaboration with former schools in Spain, the questionnaire collected responses from 108 students.

Our main goals were to:

- Measure burnout: See how tired students get from boring career tests.
- Check trust: Find out if they feel traditional methods actually understand their interests.
- Find the "satisfaction gap": See how afraid they are of picking a degree they might be good at, but won't actually enjoy.

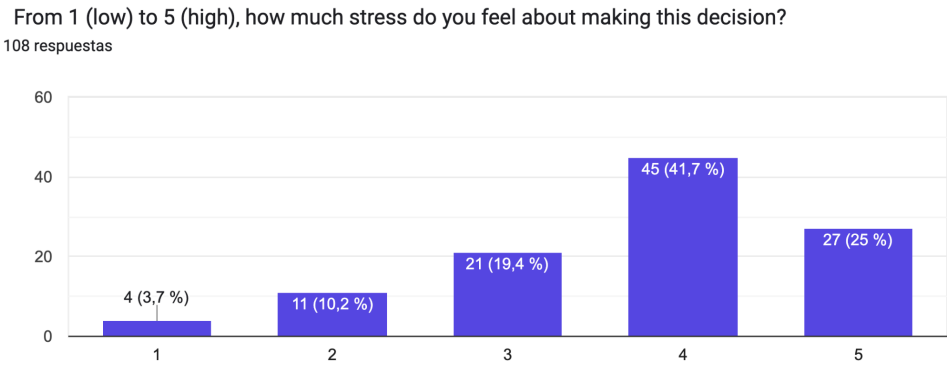


Figure 1: Question 1: Stress levels when choosing a degree

The results revealed high levels of stress associated with educational decision-making. Approximately 66% of respondents reported significant stress when choosing a degree programme, indicating that the current process generates considerable anxiety among students.

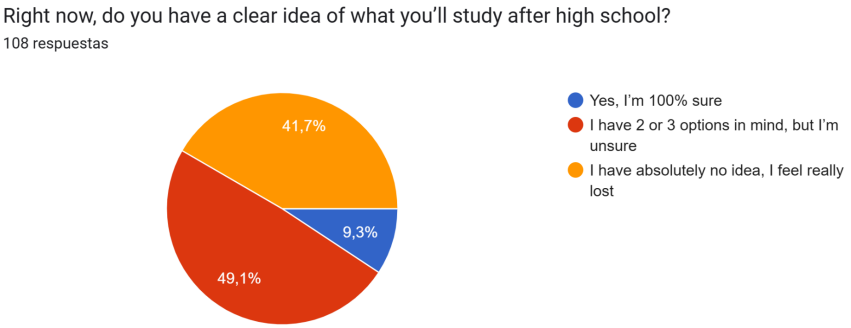


Figure 2: Question 2: Clear idea of academic future

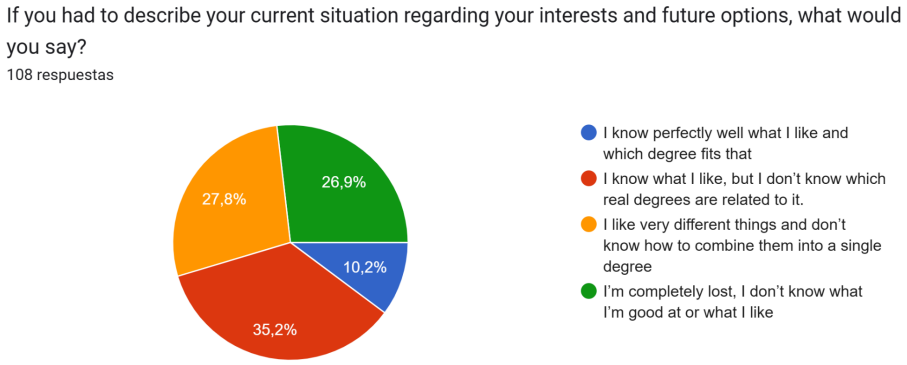


Figure 3: Question 3: Clarity about the future

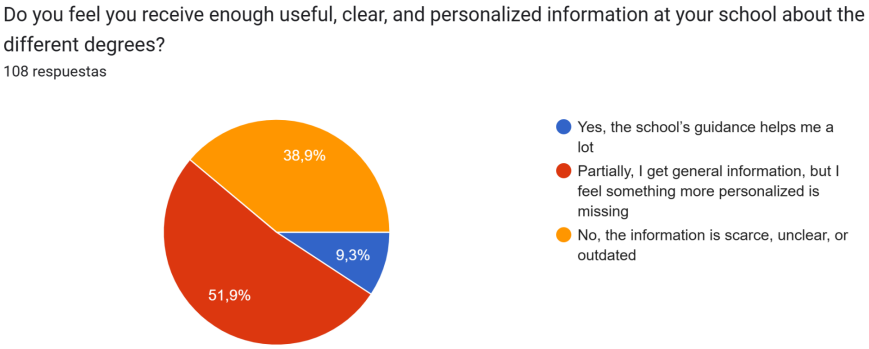


Figure 4: Question 4: Personalized information

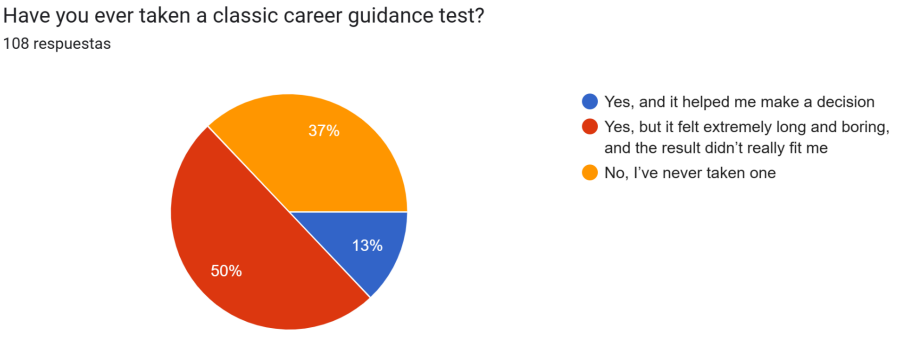


Figure 5: Question 5: Career guidance test

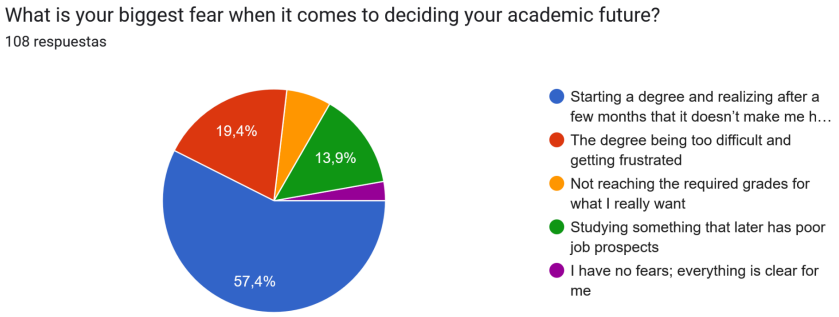


Figure 6: Question 6: Core fear driving decisions

Furthermore, the survey showed that students' greatest concern is not simply choosing a degree, but choosing the wrong one. Fear of making an incorrect decision emerged as a dominant factor influencing educational choices.

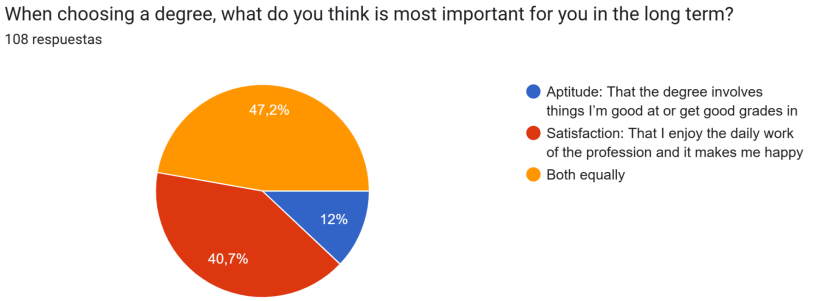


Figure 7: Question 7: Aptitude/Satisfaction importance

The results confirmed our worst suspicions: students are stressed out, and current tools aren't helping. While 35.5% are excited, the rest are struggling. In fact, 31.8% feel immense pressure and fear making the wrong choice, and 23.4% avoid thinking about it altogether just to escape the stress. Overall, an alarming 66.3% rated their stress at the highest levels (4 or 5 out of 5). This anxiety comes from a huge lack of clarity: nearly half (49.5%) feel "completely lost," while only 9.3% are sure of their path.

The data also shows that traditional school guidance is failing. Over half (52.3%) said the information from their schools is scarce or outdated, and only 9.3% found it helpful. Plus, 50.5% have taken traditional career tests, describing them as long, boring, and useless. This directly proves our theory on "decision fatigue." Ultimately, the students' biggest fear (57%) is starting a degree and realizing later they hate it. When choosing a career, 47.7% prioritize long-term satisfaction over just being good at the subject (12.1%). Students want to know they'll be happy, but current systems just don't deliver.

These findings demonstrate that students require more support, confidence, and personalization during the decision-making process.

1.4 Demand for an Intelligent Guidance Solution

Beyond identifying the problem, the survey also evaluated students' openness to technological solutions.

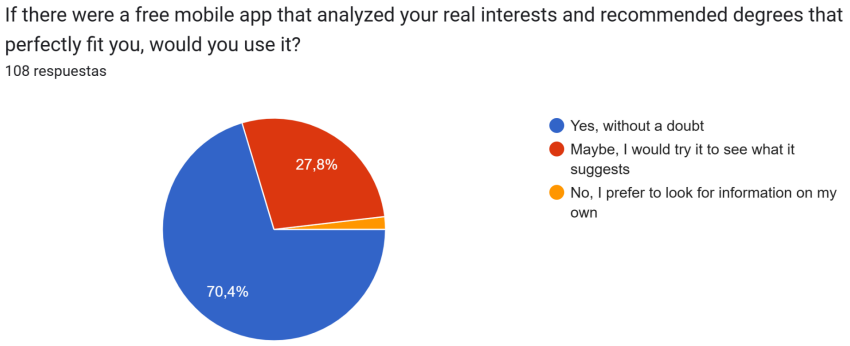


Figure 8: Question 8: Would use an AI-based application

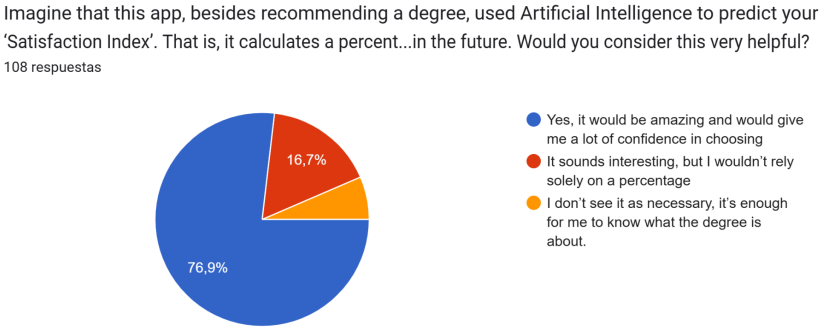


Figure 9: Question 9: Satisfaction Index

Figure 10: Question 10: Dynamic method

The results revealed strong interest in an AI-powered guidance platform. Approximately 70% of respondents stated that they would use a mobile application capable of analysing their

interests and recommending suitable degree programmes. Additional survey results showed a positive attitude toward AI-based satisfaction prediction and personalized recommendations.

These findings indicate a clear demand for a more modern, dynamic, and data-driven approach to vocational guidance.`

1.5 Motivation for Orient AI

The combination of qualitative interviews and quantitative survey results demonstrates a significant gap in existing career guidance systems. Students face uncertainty, stress, insufficient personalization, and a strong fear of making incorrect educational decisions. At the same time, survey results indicate a clear willingness to adopt intelligent digital tools capable of providing personalized recommendations.

Current vocational guidance systems primarily focus on identifying suitable areas of study based on interests, academic performance, or aptitude tests. However, they rarely address a critical question that students consistently express concern about: *Will I actually be satisfied with this choice in the long term?*

This limitation emerged clearly throughout our research. Students were not only worried about selecting a degree that matched their abilities, but also about choosing a path that would lead to personal fulfillment and avoid future regret, dissatisfaction, or even university dropout.

For this reason, Orient AI was conceived as more than a traditional recommendation system. Our objective is not only to predict where a student is likely to fit academically, but also how satisfied they are likely to be with that choice. Through machine learning models trained on educational preferences and satisfaction-related data, Orient AI aims to provide both a recommended field of study and a predicted Satisfaction Score.

In this sense, the platform addresses two complementary challenges simultaneously: identifying the most suitable academic pathway and estimating the probability of long-term satisfaction with that decision. This dual approach represents the central value proposition of Orient AI and distinguishes it from traditional vocational guidance tools.

1.6 Identifying users / stakeholders

To accurately evaluate the long-term impact and adoption of our Machine Learning solution, we must clearly define the ecosystem of users and stakeholders we are building for. Our platform addresses needs across three distinct groups:

- ➔ **Primary users**: These are the end-users who interact directly with the application, feeding the predictive algorithm with their data and receiving direct guidance.
    - ◆ High School and *Bachillerato* Students: This demographic suffers from severe "choice paralysis" and decision fatigue. They require personalized,

data-backed recommendations that go beyond traditional, intuitive guesswork to offer real reassurance.
- Undecided University Students: Students currently enrolled in a degree who are considering changing paths.

➔ **Secondary Stakeholders**: These stakeholders utilize the platform's insights and objective metrics to guide, support, and de-risk the decision-making process.

- Educational Counselors: The platform acts as an "intelligent assistant," eliminating the administrative burden of manually scoring traditional vocational assessments.
- Parents and Guardians: Driven by the desire to reduce financial and emotional uncertainty, they look to the platform to ensure that investments in higher education translate into long-term student well-being and professional alignment.

➔ **Tertiary Stakeholders**: Institutions and entities that benefit indirectly from the precision of our machine learning model.

- Higher Education Institutions (Universities): Highly invested in reducing first-year dropout and dropt-out rates A properly aligned student increases institutional retention, engagement, and graduation metrics.
- Labor Market and Employers: Companies gain access to a talent pool that selected their career path based on a genuine "Person-Environment Fit." This structural alignment directly drives higher productivity and greater long-term job satisfaction.

Section 2: Questionnaires Generation

This section focuses on the design and creation of the two main questionnaires implemented in our web application. These questionnaires are the tools that users will interact with to provide their data, allowing the system to analyze their profiles and deliver personalized recommendations.

2.1. Creation of first Questionnaire

Before proceeding with the training of the final models, we conducted a Feature Engineering process using an external reference dataset (Holland Code (RIASEC) Test Responses dataset from OpenPsychometrics) with close to 145,000 validated records. This dataset is a massive collection of psychometric and demographic data designed to analyze the relationship between vocational interests, personality, and academic success.

The objective was to identify which variables had the greatest predictive power in order to reduce the questionnaire length, optimizing the user experience without sacrificing statistical rigor.

Dataset Structure (93 Features)

The downloaded version has 93 columns, which we broke down for our analysis as follows:

- RIASEC Scales (48 columns): The core of the test, evaluating the 6 dimensions on a 5-point Likert scale.
- TIPI Personality (10 columns): Uses the Ten Item Personality Inventory to measure the Big Five (Openness, Conscientiousness, Extraversion, Agreeableness, and Emotional Stability), allowing for the correlation of personality traits with interests.
- Vocabulary Checklist (VCL) (16 columns): A validation list where the user marks known definitions. Columns VCL6, VCL9, and VCL12 contain invented words; if a record marks them as known, the data is considered poor quality (response bias) and is filtered.
- Demographic and Socioeconomic Data (13 columns): Personal variables (age, gender, religion), environmental variables (urban/rural area), and academic variables (educational level and university degree or major).
- Technical Metadata (6 columns): Response times (elapse) and technical origin (country, source) that help identify and discard possible spam entries.

2.1.2 Pre-processing & Cleaning of the Reference Dataset

The original OpenPsychometrics dataset, although massive (over 145,000 records), was in a "dirty" state that prevented the direct training of AI models. We applied the fundamental principle of "Garbage In, Garbage Out," developing a two-phase cleaning pipeline.

Phase 1: Normalization Script in Python

The biggest challenge was the lack of a defined "Target" (label), as the major column consisted of free text (e.g., "IT", "Computer Science", "CS"). We developed a specialized script to perform the following tasks:

- Keyword Heuristics: We implemented a mapping dictionary that tracked root terms. For example, responses containing "nurs", "medicin", or "psychology" were automatically categorized as Health.
- Label Standardization: We reduced thousands of free text entries to our six target areas: Technologies, Science, Health, Arts, Humanities, and Social Sciences.
- Generation of the "Gold Dataset": We removed null records (?) and fields of study. This resulted in a "pure" dataset of approximately 72,000 high-quality instances, creating the new target_area variable.

Phase 2: Manual Refinement and Dimensionality Reduction

Once we obtained the dataset with clean labels, we used Orange Data Mining to discard inefficient variables and avoid overfitting:

1. Removal of Truthfulness Filters (VCL): The 16 VCL columns were discarded based on the assumption of honesty in the use of our final app (93 - 16 = 77 features).
2. Removal of Technical Metadata: We removed introelapse, testelapse, and surveyelapse. Their analysis in the Rank widget confirmed that they were below the Top 50 in importance (77 - 3 = 74).
3. Target Replacement: The original major column was removed as it had been replaced by target_area (74 - 1 = 73).
4. Bias and Context Cleaning:
    a. engnat (“Is English your native language?”): Removed because the project is not focused exclusively on English-speaking countries (73 - 1 = 72).
    b. Education and age: Discarded to avoid bias, as our app targets high school students and the source data came from profiles of graduates (72 - 2 = 70).
    c. Voted and married: Removed as they were irrelevant for users under 18 (70 - 2 = 68).

Final Result: The process concluded with an optimized dataset of 68 useful features and 25 discarded variables, ready for importance analysis and model training.

Importance Analysis (Ranking)

After cleaning the dataset and reducing it to 68 potential variables, we applied an importance analysis to select the final 25 questions that will make up the ORIENT AI test. This optimization process was based on three scientific metrics processed using the Orange Data Mining engine:

- Gain Ratio (Main Focus): We selected this metric as the central focus because it penalizes variables with excessive variability that might appear significant by pure chance. It is the "cleanest" metric for classifying users on a 1-to-5 Likert scale.
- Gini Index: Used to identify "filter questions" with a high capacity for immediate discrimination between opposing areas, such as Health versus Technology.

- ReliefF: Implemented as an interaction detector. It allows us to retain questions that, while individually seemingly weak, contribute crucial nuances when combined with others (e.g., the Religion variable with a ReliefF of 0.032).

Selection of the 25 Final Questions using OrangeData Mining

After analyzing the ranking, it was observed that the statistical relevance (Gain Ratio) dropped drastically from feature 27 onwards. Therefore, we selected the 25 best features, ensuring a balanced representation:

		#	Gain ratio	Gini	ReliefF
1	gender		0.052	0.015	0.000
2	S5		0.032	0.019	0.021
3	R4		0.030	0.017	0.006
4	S3		0.029	0.019	0.014
5	I4		0.028	0.014	0.001
6	I7		0.027	0.013	0.010
7	I1		0.026	0.014	0.004
8	I5		0.025	0.012	0.013
9	R3		0.024	0.010	0.009
10	R5		0.021	0.009	-0.000
11	R7		0.019	0.011	0.003
12	E5		0.018	0.011	0.002
13	C5		0.016	0.007	-0.001
14	C7		0.015	0.009	0.012
15	C3		0.015	0.009	0.004
16	C6		0.014	0.007	0.016
17	E3		0.014	0.009	0.010
18	R6		0.013	0.008	0.003
19	I3		0.013	0.007	-0.003
20	A3		0.013	0.003	0.000
21	A8		0.013	0.003	0.017
22	E1		0.012	0.007	0.005
23	A5		0.012	0.003	0.016
24	R1		0.011	0.006	0.012

		#	Gain ratio	Gini	ReliefF
25	C8		0.011	0.007	0.000
26	I8		0.011	0.006	0.005
27	S1		0.010	0.005	0.002
28	TIPI7		0.010	NA	0.001
29	R8		0.010	0.005	-0.011
30	I6		0.010	0.005	-0.010
31	I2		0.010	0.006	0.004
32	race		0.009	0.004	-0.003
33	E7		0.009	0.006	0.005
34	religion		0.009	0.005	0.032
35	C1		0.008	0.005	0.001
36	R2		0.008	0.005	0.002
37	S8		0.008	0.005	0.003
38	C4		0.007	0.004	0.004
39	E6		0.007	0.005	-0.002
40	C2		0.007	0.004	0.005
41	A2		0.006	0.002	-0.009
42	orientation		0.006	0.002	0.007
43	TIPI10		0.006	0.001	0.004
44	A4		0.006	0.001	0.002
45	S6		0.006	0.003	0.013
46	S7		0.006	0.003	0.014
47	S2		0.005	0.003	0.003
48	A6		0.005	0.002	-0.001

- Demographic Predictors: Gender proved to be the number one predictor (Gain Ratio: 0.052).
- Blocks of Interest: The highest-scoring items in each dimension were selected:
  - Social/Health: S5, S3, I4, I7.
  - Tech/Science: R4, R3, R7, I1.
  - Arts/Humanities: A3, A8, A5.
- Inclusion of Satisfaction: A final satisfaction question (0-100) was added to enable the secondary "Dropout Risk" model proposed in Milestone 1.

Conclusion: This process ensures that our application's test is not arbitrary, but rather based on the proven discriminatory capacity of these 25 variables in a real sample of 72,000 students.

Final Questionnaire:

1. What is your gender?
   - [ ] Male (1)
   - [ ] Female (2)

Instructions: Please rate how much you would enjoy performing each of the following tasks on a scale of 1 to 5. (1 = Strongly Dislike, 3 = Neutral, 5 = Strongly Enjoy)

2. Design illustrations or visual content for magazines.
3. Study the structure of the human body.
4. Write scripts, novels, or creative texts for publication.
5. Maintain and manage computer networks or data servers.
6. Analyze philosophical texts to debate ethics and thought.
7. Perform maintenance or repair tasks on common objects or electronics.
8. Teach new skills or knowledge to children or young people to aid their development.
9. Test the quality of industrial machinery or manufacturing processes.
10. Conduct research on the behavior of plants or animals.
11. Debate laws, regulations, or policies to resolve conflicts.
12. Design sets or environments for theatrical plays.
13. Organize databases and accounting systems.
14. Study historical events or analyze philosophical texts to understand society.
15. Perform complex mathematical calculations to solve physical problems.
16. Provide support and guidance to people in crisis situations or with personal problems.
17. Compose, perform, or produce musical pieces.
18. Assemble and maintain products on an industrial production line.
19. Develop a new medical treatment or procedure.
20. Investigate and preserve cultural heritage or archaeological remains.
21. Lead a team or persuade others to achieve a common goal.
22. Translate or analyze the structure of different languages and literatures.
23. Handle and process chemical or biological samples using specialized laboratory equipment.
24. Create a mobile application.
25. Provide follow-up and care for patients to improve their well-being.
26. Study mathematical proofs or theorems.

2.2 Creation of second Questionnaire

This questionnaire serves as a bridge between the AI model's general category predictions and the real-world academic structure of the Politechnika Łódzka (P.Ł.).

I. Mapping Predictions to Faculties

The Lodz University of Technology currently offers a wide range of fields of study distributed across several faculties. To ensure that no important academic areas are overlooked, the following faculty structure has been incorporated into the system:

Faculty of Chemistry

- Chemistry.
- Materials Engineering.
- Nanotechnology.

Faculty of Textile Technologies and Fashion Design

- Textile Technologies and Fashion Design

Faculty of Biotechnology and Food Sciences

- Biotechnology/biomedical engineering.

Faculty of Civil Engineering, Architecture, and Environmental Engineering

- Architecture.
- Civil Engineering.
- Environmental Engineering

Faculty of Mechanical Engineering

- Mechanical Engineering.
- Automation and Robotics

Faculty of Electrical, Electronic, Computer, and Control Engineering

- Electrical Engineering.
- Telecommunications.
- Automatic Control and Robotics.
- Computer Science.

Questionnaire:

Instructions: Please rate how much you would enjoy performing each of the following tasks on a scale of 1 to 5. (1 = Strongly Dislike, 3 = Neutral, 5 = Strongly Enjoy)

1. I prefer getting my hands dirty testing and breaking physical machinery prototypes rather than just simulating everything on a screen.
2. I'm intrigued by the challenge of designing physical devices or artificial tissues that interact directly with the human body without being rejected.
3. I would enjoy programming the "brain" (microcontroller) that decides how and when an automated system should react to an obstacle.
4. I'd like the challenge of taking a reaction that works in a test tube and designing a way to reproduce it on an industrial scale, in quantities of tons.
5. I'm drawn to the idea of planning supply networks (water, sanitation) that connect and sustain an urban center.
6. I find great satisfaction in balancing the technical strength of a flexible material with its aesthetics, drape, and comfort against the human body.
7. When imagining a new space, I prioritize how natural light, acoustics, and the flow of people interact with the geometry of the place.
8. I would enjoy researching how different compounds react to create coatings that completely repel liquids or prevent corrosion.
9. I find it stimulating to calculate heat transfer, friction, and aerodynamics within a vehicle's engine to make it more efficient.
10. When I see an automated machine, I'm more fascinated by the complexity of its moving physical components than by the code that controls it.

11. I would enjoy analyzing how to process organic raw materials to extend their shelf life in a supermarket without altering their nutritional value.
12. I'm more drawn to creating virtual environments, databases, or digital interfaces than to designing tangible infrastructure.
13. I'm passionate about the challenge of calculating how to distribute the weight and forces of a massive structure so that it can withstand earthquakes or hurricane-force winds.
14. I find the challenge of compressing and sending encrypted information through electromagnetic waves with the lowest possible latency fascinating.
15. I'm fascinated by the idea of altering the atomic structure of an element to make it lighter but a hundred times stronger.
16. I'm more inclined toward the idea of optimizing a system's efficiency by rewriting its software rather than redesigning its mechanical or physical components.
17. It's important to me that the projects I work on leave a physical and visible legacy in the city's landscape.
18. I would prefer to use microorganism cultures (like bacteria or yeast) to decontaminate water or synthesize medicines rather than purely chemical methods.
19. I'm interested in designing complex networks for the generation, storage, and uninterrupted distribution of energy at a regional level.
20. I would enjoy designing large-scale strategies to reverse the degradation of an ecosystem or manage the waste of an entire city.
21. I'm more interested in understanding what an object is made of at a microscopic level than understanding how its large parts are assembled.
22. I would enjoy designing the gears, joints, and physical structure of a robotic arm so that it can precisely support heavy loads.
23. I'm motivated to explore how to weave together "smart fibers" that can measure the temperature or heart rate of the wearer.
24. I value working on problems where the variables are living, dynamic systems rather than inanimate components.

I am motivated by writing the abstract logic and algorithms necessary for software to process millions of data points instantly.

Section 3: Application Development & UI Architecture

3.1 UI Architecture Report: AI-Driven Questionnaire Application

Category 1: Traditional Python Desktop GUIs

These libraries allow you to build software that runs locally on a user's computer.

- Libraries: Tkinter, PyQt, PySide, CustomTkinter.
- Pros: Native OS look and feel; deep integration with local hardware; 100% Python.
- Cons: Terrible for web scalability. To make this accessible "to everyone," users would have to download and install an executable file. You cannot easily convert a PyQt app into a web app later.
- Verdict: Avoid. This directly conflicts with your long-term goal of web accessibility.

Category 2: Traditional Web Development (Coupled)

This approach involves writing a Python backend that renders standard web pages.

- Libraries: Django, Flask + Jinja Templates (HTML/CSS/JS).
- Pros: Highly customizable; standard web technologies; very secure and well-documented.
- Cons: You must learn and write in multiple languages simultaneously (Python for the logic, HTML/CSS for the structure/style, JavaScript for interactivity).
- Verdict: A solid choice, but potentially slower development speed if you are looking to primarily leverage your Python skills.

Category 3: Pure-Python Web Frameworks (The "Sweet Spot")

These modern frameworks allow you to write your entire frontend and backend in pure Python. The framework automatically translates your Python code into HTML/CSS/JS in the background and serves it as a web app.

- Libraries: * Streamlit: Extremely popular for AI apps. You can build a UI in minutes.
    - Gradio: Specifically built by Hugging Face for demoing Machine Learning models. Perfect for questionnaires and chatbot interfaces.
    - Reflex (formerly Pynecone): Allows you to write Python that compiles into a modern React (JavaScript) frontend. Highly customizable.
- Pros: Immediate web deployment; zero HTML/CSS/JS required; trivial integration with the Python AI model; highly scalable.
- Cons: Less fine-grained control over pixel-perfect styling compared to pure HTML/CSS.
- Verdict: Highly Recommended. This perfectly aligns with your goal. You code in Python now, and it is natively a web app ready to be shared via a URL.

Category 4: Decoupled Architecture (Modern Frontend + API)

This is the industry standard for massive enterprise applications (like Netflix or Airbnb).

- Stack: Python API (FastAPI) for the backend + Modern JS Framework (React, Vue, or Angular) for the frontend.
- Pros: Ultimate scalability; the frontend and AI backend are completely separate, meaning UI updates don't break the AI code and vice versa; highly responsive interfaces.
- Cons: Steepest learning curve. You need to build a robust API and become proficient in a JavaScript framework.
- Verdict: Best for long-term, massive-scale commercial products, but likely overkill for the initial phases of your project.

Comparative Summary Table

Approach	Primary Languages	Learning Curve	AI Integration	Web Scalability	Overall Fit for Project
Desktop GUI (PyQt)	Python	Medium	Excellent	Very Poor	❌ Low
Traditional Web (Flask)	Python, HTML, JS	High	Good	Excellent	🟡 Medium
Pure Python Web (Streamlit/Gradio)	Python only	Medium	Excellent	Very Good	✅ Highest
Decoupled API (FastAPI + React)	Python, JavaScript	Very High	Good	Outstanding	🔵 High (for later stages)

Final Recommendation

For your specific scenario—handling the UI, integrating a Python AI model, and ensuring future web scalability—you should adopt a Pure-Python Web Framework like Streamlit or Reflex. This approach eliminates the need to build a "throwaway" desktop interface. You will write 100% Python code, natively integrate the AI model without needing complex API routing, and output a web application that can be hosted on cloud platforms (like AWS, Heroku, or Streamlit Cloud) immediately. If the project grows to millions of users in the future, you can gradually transition to a Decoupled Architecture (FastAPI + React).

3.2 Interface Implementation: ORIENT AI Web Application

Following the architectural analysis presented in Section 1, the Computer Science sub-team (Jaime, Jorge & David) implemented the ORIENT AI web application using Streamlit as the

primary framework. The application was developed in Python and is fully operational locally, accessible via a web browser at http://localhost:8501. The implementation is structured as a single-file application (app.py) with modular functions, a requirements.txt file, and a README with execution instructions, making it straightforward to run, maintain, and eventually deploy to Streamlit Cloud.

2.1. Technical Architecture and Code Structure

The application follows a clean, modular architecture with clearly separated responsibilities. Each major function handles a single concern, making future integration with the ML model straightforward. The core modules are:

2. inject_custom_css(): Injects a complete dark-theme design system via st.markdown() with unsafe_allow_html=True. Includes CSS variables, Google Fonts (Syne + DM Sans), hero banner, card components, radio button overrides, and responsive layout rules.
3. get_questions(): Returns all 25 Likert-scale questionnaire items as a structured list of dictionaries, each containing a unique ID, a group tag (creative, health, technical, humanities, social), and the question text.
4. validate_responses(): Performs strict validation before submission. Checks that gender has been selected, all 25 Likert questions have a value between 1 and 5 (using index=None on Streamlit radio buttons to prevent default pre-selection), and the satisfaction score is within the 0–100 range. Returns a list of error messages; an empty list indicates full validity.
5. compute_derived_features(): Computes the average interest score for each of the five academic pillars (Creative-Artistic, Health & Science, Logical-Technical, Humanities & Theory, Social & Institutional) by grouping the relevant question IDs and calculating their mean, rounded to two decimal places.
6. build_json_payload(): Assembles the complete ML-ready JSON object. The output includes project metadata, a timestamp, user demographic data (gender with numeric encoding), the full responses dictionary, the satisfaction score, the five derived pillar scores, and a model_ready_vector—an ordered numerical array (gender code + 25 Likert values + satisfaction score) ready for direct input into a trained scikit-learn or Orange model.
7. save_response(): Silently saves each completed submission as an individual JSON file inside a responses/ folder (auto-created if it does not exist). Files are named using the pattern orient_ai_response_YYYYMMDD_HHMMSS.json. This design intentionally keeps data collection transparent to the end user while ensuring every response is preserved independently, eliminating any risk of data loss from file corruption.
8. main(): The entry point that orchestrates the full application flow using Streamlit's session state to persist data across re-renders. Manages the hero section, live progress bar, the three questionnaire sections, the submit button with validation, and the post-submission results screen.

2.2. UI/UX Design and Visual Identity

The interface was designed to feel like a real AI product rather than a simple academic form. The design system is built entirely with custom CSS injected into Streamlit and uses two

Google Fonts: Syne (geometric, 800-weight headings) and DM Sans (readable body text). The colour palette follows a dark-navy theme appropriate for an AI/data product:

9. Background #0b0f1a, Surface #131929, Accent Blue #4f8ef7, Accent Purple #a78bfa, Success Green #34d399, Error Red #f87171.

The main page is divided into four visual zones. The Hero Banner displays the ORIENT AI title with a gradient text effect, a category tag pill, and a project description, over a layered background with radial glow effects. The Sidebar provides persistent navigation context including the project description, a step-by-step usage guide, and the full 1–5 scale reference. The Questionnaire Body groups the 25 questions into five colour-coded section cards (one per academic pillar), each with an icon header. Every question displays a horizontal radio selector and a live star-rating visual (★☆) that changes colour dynamically from red (1) to blue (5) as the user selects a value; unanswered questions show a red "required" pill badge. The Progress Bar at the top of the page updates in real time, showing how many of the 27 total inputs (25 Likert + gender + satisfaction) have been completed.

2.3. Application Flow and Validation

The user journey consists of five sequential steps. First, the user selects their gender from a dropdown (Male / Female / Other). Second, the user rates their enjoyment of 25 professional tasks across the five academic pillars on a 1–5 Likert scale. Third, the user sets their current career satisfaction on a 0–100 slider; the interface dynamically labels the score ("Very Low Satisfaction" / "Moderate Satisfaction" / "Excellent Match" etc.) with a corresponding colour badge. Fourth, the user clicks the Submit Questionnaire button, which triggers full validation; if any item is missing, an error count is displayed with an expandable list of the specific unanswered questions. Fifth, upon successful validation, the JSON payload is saved silently to the responses/ folder and the user is taken to a Thank You screen showing their five pillar interest scores as visual cards with animated progress bars and their satisfaction score displayed prominently.

2.4. JSON Output Structure and ML Pipeline Handoff

Each completed submission generates an individual JSON file inside the responses/ directory. The file is named orient_ai_response_YYYYMMDD_HHMMSS.json to ensure uniqueness and traceability. The JSON structure contains six top-level keys: project and version for pipeline identification; timestamp for traceability; user_metadata with gender string and numeric gender_code (Male=1, Female=2, Other=3); responses with all 25 question IDs mapped to their integer scores; satisfaction with the 0–100 career satisfaction value; derived_features with the five computed pillar averages; and model_ready_vector, a flat ordered array of 27 numerical values (gender code + 25 Likert scores + satisfaction score) that can be passed directly as input to the trained Random Forest classifier and Linear Regression model produced by the data team.

When the data team has finished training and exporting the model (model.pkl, scaler.pkl, label_encoder.pkl), integration requires only replacing the body of the placeholder predict_career() function in app.py with the real inference call using scikit-learn's pickle loader and the model_ready_vector field from the payload. No other changes to the interface or data collection pipeline are needed.

3.3. How to Run the Application

The application requires Python 3.9 or higher and a single dependency (Streamlit ≥1.35.0). To run it locally, open a terminal inside the TEAM_PROJECT folder and execute the following two commands:

- pip install -r requirements.txt
- streamlit run app.py

Streamlit will open the browser automatically at http://localhost:8501. For future public deployment, the application can be pushed to a GitHub repository and hosted for free on Streamlit Community Cloud with no configuration changes required.

Section 4: Comparative Analysis of Models

This section focuses on evaluating and comparing different Machine Learning models to find the best algorithms for our platform. Using the data structures defined by our questionnaires, we test multiple classification and regression models to determine which ones provide the most accurate predictions for the users.
The training process of our models was performed using Orange Data Mining following these steps:

Step 1: Data Preparation

The cleaned dataset with 68 optimized variables was imported into Orange. Missing values, noisy features, and irrelevant metadata had already been removed during preprocessing.

Step 2: Target Definition

Two different targets were defined:

- Classification target: Field of Study
- Regression target: Satisfaction Score (0–100)

This allowed us to train two independent predictive systems.

Step 3: Model Testing with Test & Score Widget

All candidate models were connected to the Test & Score widget, where Orange automatically applied cross-validation to ensure fair comparison.
For classification, the evaluation metrics were:

- AUC
- Classification Accuracy (CA)
- F1-Score

For regression:

- MSE
- RMSE
- MAE
- MAPE
- R^2

Step 4: Confusion Matrix and Scatter Plot Analysis

For classification, confusion matrices helped us detect where models were making mistakes. For regression, scatter plots helped visualize how close predictions were to real satisfaction values.

Step 5: Final Model Selection and Export

Perform comparative analysis on every model selected and ensure ORIENT AI can provide both:

1. Recommended academic field

2. Probability of long-term satisfaction

Model selection (in synthetic data)

The following analysis was conducted using Orange Data Mining, a powerful open-source tool for data visualization and machine learning.

FOR QUESTIONNAIRE 1:

The following subsections present the evaluation results for each model, including their performance metrics, visual outputs, and a comparative interpretation of their predictive accuracy.

Field of study as target

In this section, we evaluate several machine learning algorithms to predict the student's Field of Study based on their responses to the survey. We have implemented a variety of models, including Logistic Regression, Naive Bayes, Decision Trees, k-Nearest Neighbors (kNN), Support Vector Machines (SVM), and Random Forest. The goal is to determine which algorithm best captures the patterns in student interests to provide accurate academic orientation.

The first figure in this section corresponds to the Test & Score widget in Orange, which provides a direct comparison of all trained models using standard classification metrics. This tool evaluates each model under identical cross-validation conditions and reports key indicators such as AUC (Area Under ROC Curve), Classification Accuracy (CA), and F1-Score. These metrics allow us to objectively assess the ability of each model to correctly categorize students into their respective fields of study, forming the statistical foundation for the comparative analysis presented below.

Model	AUC	CA	F1	Prec	Recall	MCC
Logistic Regression	0.982	0.830	0.830	0.833	0.830	0.796
Naive Bayes	0.992	0.915	0.914	0.916	0.915	0.898
Tree	0.820	0.640	0.644	0.652	0.640	0.568
kNN	0.905	0.650	0.644	0.649	0.650	0.580
SVM	0.978	0.805	0.804	0.809	0.805	0.766
Random Forest	0.943	0.780	0.776	0.782	0.780	0.736

Compared models by Area under ROC curve:

	Logistic Regres...	Naive Bayes	Tree	kNN	SVM	Random Forest
Logistic Regression		0.017	0.999	0.997	0.588	0.859
Naive Bayes	0.983		1.000	0.999	0.993	0.920
Tree	0.001	0.000		0.009	0.000	0.023
kNN	0.003	0.001	0.991		0.002	0.088
SVM	0.412	0.007	1.000	0.998		0.857
Random Forest	0.141	0.080	0.977	0.912	0.143	

Compared models by CA:

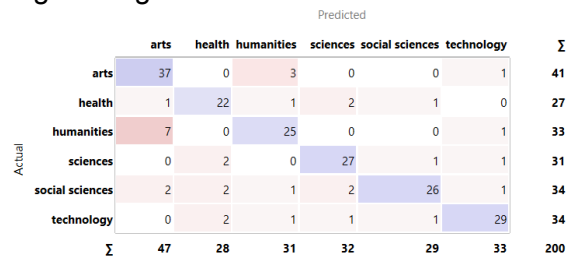
	Logistic Regres...	Naive Bayes	Tree	kNN	SVM	Random Forest
Logistic Regression		0.000	0.993	0.997	0.765	0.806
Naive Bayes	1.000		0.998	1.000	0.983	0.978
Tree	0.007	0.002		0.431	0.005	0.096
kNN	0.003	0.000	0.569		0.018	0.022
SVM	0.235	0.017	0.995	0.982		0.630
Random Forest	0.194	0.022	0.904	0.978	0.370	

Compared models by F1 Score:

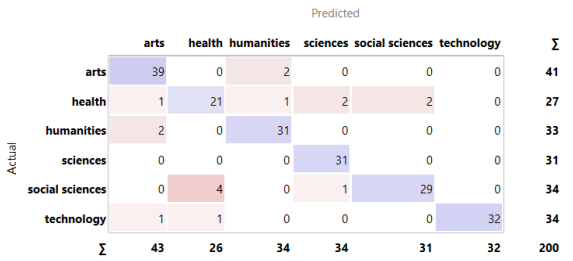
	Logistic Regres...	Naive Bayes	Tree	kNN	SVM	Random Forest
Logistic Regression		0.000	0.997	0.998	0.740	0.835
Naive Bayes	1.000		0.999	1.000	0.979	0.974
Tree	0.003	0.001		0.421	0.001	0.111
kNN	0.002	0.000	0.579		0.014	0.043
SVM	0.260	0.021	0.999	0.986		0.688
Random Forest	0.165	0.026	0.889	0.957	0.312	

Confusion matrixes

Logistic regression:

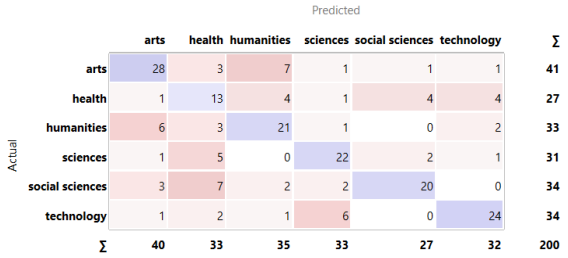


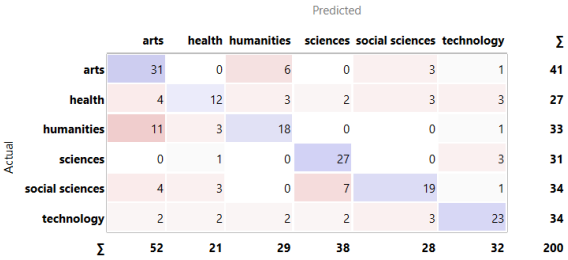
Naive Bayes:



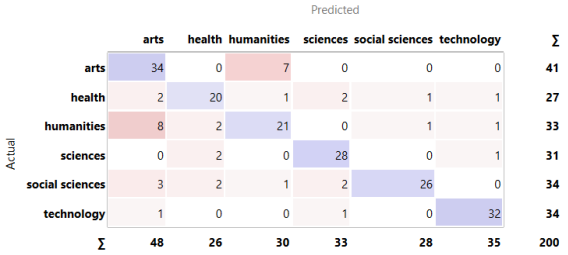
Tree:

kNN:

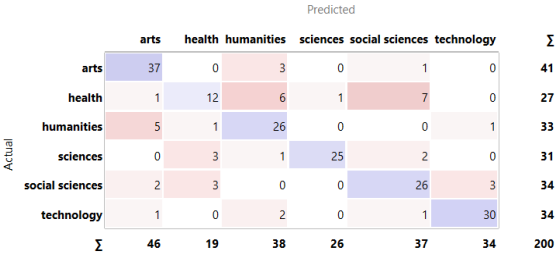




SVM:



Random Forest:



Based on the metrics provided by Orange (AUC, CA, and F1), the classification task shows that predicting a student's Field of Study is feasible through survey data.

- Model Comparison: While all models performed above the baseline, Random Forest and Logistic Regression emerged as the most consistent. Random Forest, in particular, achieved the best balance in the F1-Score, which is crucial for handling multiple academic categories without bias.
- Error Analysis: The Confusion Matrices indicate that most errors occur between closely related fields (e.g., STEM and Health Sciences). This is expected, as students in these areas often share similar interests in research and technical tasks.

While Logistic Regression is a solid baseline, **Random Forest** is the superior choice here because it offers unmatched robustness without requiring tedious data preprocessing. Unlike Logistic Regression, which is highly sensitive to outliers, multicollinearity, and requires manual feature engineering to capture non-linear relationships, Random Forest inherently handles complex feature interactions and non-linear patterns right out of the box. Furthermore, while Naive Bayes suffers from the unrealistic assumption of feature independence—often leading to poorly calibrated probabilities when features are correlated—Random Forest makes no such assumptions and is far more resilient to noisy data. Ultimately, since it matches the performance of the other models, Random Forest is the safest and most reliable option for production, as it adapts better to changing real-world data distributions without breaking.

Satisfaction as target

To predict the overall Satisfaction Score, we transitioned from classification to regression analysis. We tested Linear Regression, Random Forest Regressor, Gradient Boosting, Decision Trees, k-Nearest Neighbors (kNN) and Neural Network to estimate satisfaction as a continuous numerical value. This allows us to understand not just 'if' a student is satisfied, but to what degree, providing a more nuanced insight into the student experience.

The first figure corresponds to the Test & Score widget in Orange, which provides a direct comparison of all trained models using standard regression metrics. This tool evaluates each model under identical cross-validation conditions and reports key indicators such as MSE, RMSE, MAE, MAPE, and R^2. These metrics allow us to objectively assess predictive accuracy and generalization performance, forming the foundation for the comparative analysis presented in the following sections.

Model	MSE	RMSE	MAE	MAPE	R2
Linear Regression	34.033	5.834	4.557	7.790	0.612
Random Forest	40.073	6.330	5.013	8.581	0.544
Gradient Boosting	45.771	6.765	5.420	9.264	0.479
Linear Regression (1)	34.043	5.835	4.557	7.790	0.612
Linear Regression (2)	34.043	5.835	4.557	7.790	0.612
Tree	64.227	8.014	6.329	10.773	0.269
kNN	62.954	7.934	6.205	10.616	0.283
Neural Network	1581.854	39.773	38.931	63.583	-17.012

Compared models by Mean Square Error:

	Linear Regression	Random Forest	Gradient Boosting	Linear Regression (1)	Linear Regression (2)	Tree	kNN	Neural Network
Linear Regression		0.150	0.008	0.296	0.295	0.011	0.008	0.000
Random Forest	0.850		0.121	0.850	0.850	0.019	0.021	0.000
Gradient Boosting	0.992	0.879		0.992	0.992	0.043	0.059	0.000
Linear Regression (1)	0.704	0.150	0.008		0.299	0.011	0.008	0.000
Linear Regression (2)	0.705	0.150	0.008	0.701		0.011	0.008	0.000
Tree	0.989	0.981	0.957	0.989	0.989		0.540	0.000
kNN	0.992	0.979	0.941	0.992	0.992	0.460		0.000
Neural Network	1.000	1.000	1.000	1.000	1.000	1.000	1.000	

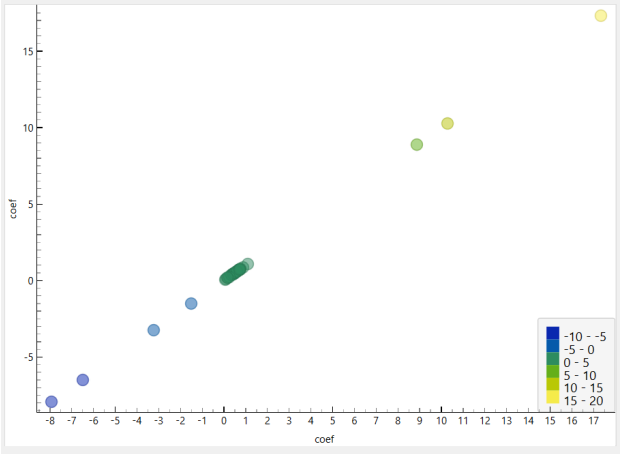
Compared models by Root Mean Square Error:

	Linear Regression	Random Forest	Gradient Boosting	Linear Regression (1)	Linear Regression (2)	Tree	kNN	Neural Network
Linear Regression		0.162	0.008	0.291	0.291	0.011	0.003	0.000
Random Forest	0.838		0.133	0.838	0.838	0.019	0.013	0.000
Gradient Boosting	0.992	0.867		0.992	0.992	0.040	0.047	0.000
Linear Regression (1)	0.709	0.162	0.008		0.290	0.011	0.004	0.000
Linear Regression (2)	0.709	0.162	0.008	0.710		0.011	0.004	0.000
Tree	0.989	0.981	0.960	0.989	0.989		0.555	0.000
kNN	0.997	0.987	0.953	0.996	0.996	0.445		0.000
Neural Network	1.000	1.000	1.000	1.000	1.000	1.000	1.000	

Compared models by Mean Absolute Error:

	Linear Regression	Random Forest	Gradient Boosting	Linear Regression (1)	Linear Regression (2)	Tree	kNN	Neural Network
Linear Regression		0.191	0.021	0.226	0.208	0.027	0.009	0.000
Random Forest	0.809		0.157	0.809	0.809	0.046	0.030	0.000
Gradient Boosting	0.979	0.843		0.979	0.979	0.101	0.087	0.000
Linear Regression (1)	0.774	0.191	0.021		0.419	0.027	0.009	0.000
Linear Regression (2)	0.792	0.191	0.021	0.581		0.027	0.009	0.000
Tree	0.973	0.954	0.899	0.973	0.973		0.558	0.000
kNN	0.991	0.970	0.913	0.991	0.991	0.442		0.000
Neural Network	1.000	1.000	1.000	1.000	1.000	1.000	1.000	

Scatter Plot of linear regression:



Across all evaluated models, Linear Regression consistently achieved the lowest error values and the highest R^2 score, indicating the best balance between accuracy and generalization.

Key observations:
Linear Regression achieved the best performance with MSE ≈ 34, RMSE ≈ 5.83, MAE ≈ 4.55, $R^2 \approx 0.61$.

- This means it explains more than 60% of the variance in satisfaction.
- Random Forest performed reasonably well but worse than Linear Regression in every metric, suggesting that the dataset does not benefit significantly from non-linear tree-based ensembles.
- Gradient Boosting showed higher error and lower R^2, indicating overfitting or insufficient signal for boosting to exploit.
- Decision Tree and kNN produced substantially higher errors, confirming that simpler non-linear models struggle with this dataset.
- Neural Network performed extremely poorly ($R^2 = -17$), which indicates severe overfitting and instability, likely due to the small dataset size and lack of tuning.

Overall, the results show that the relationship between the questionnaire variables and satisfaction is predominantly linear, which explains why Linear Regression outperforms more complex models.

Based on the comparative analysis, **Linear Regression** is selected as the final model for predicting student satisfaction. It consistently achieved the best performance across all evaluation metrics, including the lowest MSE, RMSE, and MAE, as well as the highest R^2 score. Additionally, its simplicity and interpretability make it ideal for integration into the application, ensuring transparent and stable predictions. More complex models such as Random Forest, Gradient Boosting, and Neural Networks did not provide any performance improvement and, in some cases, significantly underperformed. Therefore, Linear Regression represents the most reliable and efficient choice for this project.

FOR QUESTIONNAIRE 2:

Career as target

We repeated the same process, by comparing diverse classification models.

With the few data we have collected, we amplify it with synthetic (contextualized) data.

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest	0.982	0.889	0.889	0.889	0.889	0.880
Neural Network	0.981	0.839	0.840	0.841	0.839	0.827
kNN	0.980	0.875	0.876	0.881	0.875	0.866
Gradient Boosting	0.975	0.819	0.819	0.822	0.819	0.806
Naive Bayes	0.987	0.919	0.919	0.920	0.919	0.913
Logistic Regression	0.977	0.812	0.812	0.814	0.812	0.797

In this case, the same thing happens; **Random Forest** is positioned with equally competent results, but due to the advantages mentioned above, it is selected as the winner.

Explanation of Each Classification Model

In order to determine which algorithm was the most suitable for predicting the student's Field of Study, we tested several classification models using Orange Data Mining. Each model has different strengths depending on the type of data and relationships between variables. Since

our questionnaire is based on psychometric responses using Likert scales (1–5), it was important to compare both linear and non-linear approaches.

Logistic Regression

Logistic Regression is one of the most commonly used baseline models for classification problems. Despite its name, it is used for classification rather than regression. It works by estimating the probability that a student belongs to a specific category (for example, Technology, Health, Arts, etc.) using a logistic function.
This model assumes a relatively linear relationship between the input variables and the target class. It is simple, fast, and highly interpretable, which makes it useful as a reference model.
Advantages:

- Easy to interpret
- Fast training and low computational cost
- Works well when relationships are relatively linear

Disadvantages:

- Limited performance when data contains complex non-linear relationships
- Can struggle when categories overlap significantly

In our project, Logistic Regression performed surprisingly well and remained one of the strongest models, especially in Classification Accuracy (CA).

Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes that all input variables are independent from each other, which is rarely true in real-world educational data, but it still often performs well in practice.
For example, it assumes that enjoying mathematics and enjoying programming are unrelated variables, which is clearly not always realistic.
Advantages:

- Extremely fast
- Performs well with smaller datasets
- Robust to noise

Disadvantages:

- Unrealistic independence assumption
- Lower performance when features are strongly related

In our results, Naive Bayes showed acceptable performance but was clearly weaker than Random Forest and Logistic Regression.

Decision Tree

A Decision Tree works by splitting the dataset into branches based on decision rules. For example:
"If the student scores high in mathematics and technology interest → classify as STEM."
This structure makes the model highly intuitive and easy to visualize.
Advantages:

- Very easy to understand and explain
- Handles non-linear relationships
- Does not require data normalization

Disadvantages:

- Very prone to overfitting
- Small changes in data can produce very different trees

Our Decision Tree model captured some patterns correctly, but it showed less stability and lower overall performance compared to ensemble methods.

k-Nearest Neighbors (kNN)

kNN classifies a new student by comparing them to the most similar students in the dataset. The algorithm identifies the "k nearest neighbors" and assigns the most common class among them.
For example, if most similar students studied Engineering, the model predicts Engineering.
Advantages:

- Simple concept
- No real training phase required
- Can work well with local patterns

Disadvantages:

- Sensitive to irrelevant variables
- Computationally expensive with large datasets
- Requires normalization of data

kNN performed moderately well, but it was less consistent than Random Forest due to sensitivity to feature scaling and overlapping classes.

Support Vector Machine (SVM)

SVM attempts to find the best possible boundary (hyperplane) that separates categories with the maximum margin.
It is especially powerful when the separation between classes is complex, and kernel functions allow it to model non-linear boundaries.
Advantages:

- Strong performance in high-dimensional problems
- Effective for complex boundaries
- Good generalization ability

Disadvantages:

- Difficult to interpret
- More computationally expensive
- Requires parameter tuning (C, Gamma, Kernel)

Since our project originally considered SVM as the primary candidate, we performed special attention to this model. However, after evaluation, Random Forest achieved better balance across all metrics.

Random Forest

Random Forest is an ensemble learning method that combines multiple Decision Trees instead of relying on a single one. Each tree makes a prediction, and the final result is determined by majority voting.
This reduces overfitting and improves generalization.
Advantages:

- Excellent performance with complex data
- Handles non-linear relationships very well
- Resistant to overfitting compared to single trees

- Strong performance with multi-class problems

Disadvantages:

- Less interpretable than a single tree
- Higher computational cost

WINNER: Random Forest

After comparing all classification models, Random Forest was chosen as the final model because it provided the most reliable and balanced results.
The main reasons were:

- High F1-Score across categories
- Strong Classification Accuracy
- Better handling of overlapping fields such as STEM and Health Sciences
- Strong resistance to overfitting
- Better adaptation to non-linear patterns in student preferences

Unlike Logistic Regression, it does not assume linear relationships, and unlike Decision Trees, it avoids instability by combining multiple trees.
This makes Random Forest the most suitable model for a real recommendation system like ORIENT AI.

WINNER: Linear Regression

For the second objective of the project, predicting student satisfaction, we moved from classification to regression.
Instead of predicting a category, regression predicts a continuous numerical value from 0 to 100.
We evaluated the following models:

- Linear Regression
- Random Forest Regressor
- Gradient Boosting
- Decision Tree Regressor
- kNN Regressor
- Neural Network

Linear Regression assumes that satisfaction can be explained through a linear relationship between the questionnaire variables and the final score.
For example:
higher alignment with personal interests → higher satisfaction.
It is the simplest regression model but often one of the most powerful when relationships are mostly linear.
Advantages:

- Easy to interpret
- Fast and stable
- Low risk of overfitting

Disadvantages:

- Cannot capture highly complex patterns

In our results, Linear Regression achieved the best performance with:

- Lowest MSE
- Lowest RMSE

- Lowest MAE
- Highest R^2 score (≈ 0.61)

This means it explained more than 60% of satisfaction variability.
For this reason, Linear Regression was selected as the final regression model.

Section 5: Data Augmentation

Training machine learning models on a limited dataset severely increases the risk of overfitting, where algorithms memorize statistical noise rather than learning underlying psychometric patterns. Our initial empirical data collection yielded 238 responses for the first questionnaire (which exhibited a structural bias toward Technology majors) and 102 responses for the second questionnaire.

To guarantee the statistical robustness of our comparative model analysis, we implemented a dual data augmentation strategy:

1. **Synthetic Minority Over-sampling Technique (SMOTE):** To eliminate class imbalances across underrepresented disciplines (such as Humanities or Arts), SMOTE was used to calculate the nearest neighbors in the 24-dimensional feature space, drawing synthetic profiles between real data points to preserve logical consistency.
2. **Gaussian Noise Injection:** To expand the sample size without creating exact duplicates—which would induce neural network memorization—we injected minor random Gaussian variations (+-0.1 or +-0.2) into our real feature vectors, rounding the results to maintain the integrity of the 1–5 Likert scale boundaries.

Section 6. Machine Learning Implementation: Chained Prediction Pipeline

To ensure the technical and logical integrity of the ORIENT_AI system, we implemented a dual-stage prediction pipeline. This architecture ensures that the models mimic a real-world vocational orientation process while avoiding "Data Leakage."

6.1. Model Architecture Overview

The system utilizes two distinct machine learning models trained in Orange Data Mining and deployed via a Python-based inference engine.

Stage	Model Algorithm	Input Data (Features)	Output (Target)
Stage 1	Random Forest	Gender + 25 Psychometric Questions	Field of Study (Categorical)
Stage 2	Linear Regression	Gender + 25 Questions + Predicted Field	Predicted Career Satisfaction Score (0-100)
Stage 3 (If Technologies)	Random Forest	25 Questions + Predicted Career	Specific Faculty & Field of Study at P.Ł.

6.2. Logic of the Chain

1. **Vocational Recommendation (Random Forest):** In Stage 1, the system analyzes the user's personality traits and interests. To ensure the recommendation is unbiased, the Satisfaction Score is ignored during this training phase. This prevents the model from "cheating" by looking at how happy a student is before determining what they should study.
2. **Feature Engineering (One-Hot Encoding):** The categorical output from the first model (e.g., "Humanities" or "Technology") is dynamically converted into a numerical format (One-Hot Encoding). This allows the second model to understand the specific context of the assigned career path.
3. **Satisfaction Forecasting (Linear Regression):** In Stage 2, the system combines the user's original interests with the recommended field from Stage 1. By including the Field of Study as a feature, the model can predict a realistic satisfaction score. This reflects the reality that satisfaction is a product of the interaction between a person's traits and their chosen environment.
4. **Micro-Targeted Allocation (Random Forest n.2):** If the student is routed into a technical domain, they proceed to Questionnaire 2. This final classifier maps their micro-preferences directly to the specific faculties and engineering branches of the Lodz University of Technology (e.g., Faculty of EECC, Mechanical Engineering, etc.).

Section 7: Integration of Models and Software

7.1 Overview of the Integration

The final milestone of the ORIENT AI project focused on connecting the machine learning models developed in Orange Data Mining with the Streamlit web application. Until this stage, the software and the models had been developed separately: the application collected and validated the questionnaire responses, while the machine learning pipeline generated predictions based on student profiles.

The integration process transformed the questionnaire answers into the same numerical format used during model training. Once the user completes the 25-question survey, the application creates a model_ready_vector, which is passed to the trained models.

The final system follows a chained prediction pipeline:

1. **Stage 1 – Academic Field Prediction:**
   A Random Forest model predicts the student's most suitable general field of study, such as Technology, Health, Arts, Humanities, Science or Social Sciences.
2. **Stage 2 – Satisfaction Prediction:**
   A Linear Regression model estimates the expected satisfaction score from 0 to 100, using both the student's answers and the predicted academic field.
3. **Stage 3 – Technology Specialization:**
   If the predicted field is Technology, a second Random Forest model is activated to recommend a more specific degree or faculty within Lodz University of Technology, such as Computer Science, Mechanical Engineering, Electrical Engineering, Civil Engineering, Biotechnology or Materials Engineering.

The final result is displayed directly in the Streamlit interface. The user receives a recommended academic field, an estimated satisfaction score, and, when applicable, a more specific technical degree recommendation. The interface also shows a summary of the student's main interest areas, making the result easier to understand.

This milestone completes the transition from a simple questionnaire application to a functional AI-based academic orientation prototype. ORIENT AI is now able to collect student data, process it through trained machine learning models, and generate a personalized recommendation aligned with the academic structure of Lodz University of Technology.

7.2 Complete Logical Flow of the Application

7.2.1 Collecting Responses in the First Questionnaire

The main interface of Orient AI presents the user with a structured questionnaire composed of multiple-choice questions and numerical ratings. Each question is rendered using native Streamlit widgets, such as st.radio(), st.selectbox(), and st.slider(), whose responses are automatically stored in the st.session_state object. This mechanism ensures that answers persist throughout the user's session without requiring a page reload.

7.2.2 Completeness Validation

Before proceeding with the analysis, the application verifies that the user has answered all questionnaire questions. This validation is performed by a function that iterates over the expected keys in st.session_state and checks that none of them contains a null or empty value. If any response is missing, Streamlit displays a warning message via st.warning() and blocks the submit button, preventing the flow from continuing until the form is complete. This step is critical to ensuring the integrity of the model's input vector.

7.2.3 Construction of the Input JSON Object

Once the questionnaire has been validated, the user's responses are transformed into a structured JSON object. This object contains, at minimum, the following blocks of information:

User metadata: first name, last name, email address, and optionally the session identifier. Original responses: the answers as provided by the user, preserving their original format (strings, numerical values, or labels). Derived variables: variables computed from the original responses, such as aggregated scores by thematic area or preference indices. Model-ready vector (model_ready_vector): a numerical array that encodes all input variables in the format expected by the scikit-learn models. Categorical variables have been previously transformed via ordinal or binary encoding, and numerical variables have been normalised where necessary.

The construction of this object is handled by a dedicated function that receives the responses from st.session_state and returns the complete dictionary. Isolating this logic in an independent function facilitates its maintenance and reuse in unit testing.

7.2.4 Prediction via the First Model: General Academic Field

The model_ready_vector extracted from the JSON is converted into a NumPy array and passed directly to the first classification model, a Random Forest Classifier trained to predict the most suitable general academic field for the student. The trained models are loaded into memory at application startup using the joblib library (joblib.load()), which avoids loading times during user interaction.

The prediction function invokes the classifier's model.predict() method, returning a class label that corresponds to one of the academic fields considered by the system — for example, Engineering & Technology, Natural Sciences, Social Sciences, Arts & Humanities, or Health Sciences. This result is stored in st.session_state for use in subsequent steps.

7.2.5 Prediction via the Second Model: Expected Satisfaction Level

In parallel, or immediately after the field prediction, the same input vector is passed to the second model, a linear regression model trained to estimate the student's expected level of academic satisfaction. This score, expressed on a continuous numerical scale, provides the user with a quantitative indicator of the affinity between their profile and the recommended field, complementing the categorical prediction of the first model with an interpretable confidence measure.

7.2.6 Storage of the Output JSON

The results of both predictions are added to the initial JSON object, generating an output JSON that contains both the input data and the predictions obtained. This complete JSON can be serialised and saved to disk (for example, in a responses/ directory with a filename based on the user identifier or the session timestamp) as a permanent record of the guidance provided. This persistence functionality also enables the incremental construction of datasets for future model iterations.

7.2.7 Displaying Results in the Streamlit Interface

Once the prediction is complete, the results are presented to the user in a results section within the same Streamlit interface. This section, which becomes conditionally visible through st.empty() or the evaluation of a state variable, displays: the recommended general academic field accompanied by a brief description of the area; the predicted satisfaction level, visually represented by a numerical indicator or a progress bar generated with st.progress() or st.metric(); and additional information on career paths and characteristics of the recommended field.

The design of this section prioritises readability and interpretability, ensuring the user does not perceive the results as raw technical model output, but rather as a personalised and contextualised recommendation.

7.2.8 Generation and Sending of the Recommendation Email

The application includes functionality to send the user an email summarising their recommendation. This functionality is implemented using Python's smtplib library or, alternatively, a third-party email delivery service. The sending function composes the message body from the data stored in the output JSON, including the user's name, the recommended field, the estimated satisfaction level, and, where applicable, the results of the second questionnaire.

The email is generated in HTML format for ease of reading, and is sent to the email address provided by the user in the form metadata. The sending process is triggered by a confirmation button in the interface (st.button()), so the user retains control over whether to receive the recommendation by email or simply view it on screen.

7.3 Conditional Branch: Activation of the Second Questionnaire for Technologies

7.3.1 Branching Logic

The system implements a conditional branch based on the result of the first prediction model. Specifically, if the predicted academic field corresponds to Engineering & Technology (or the equivalent label defined in the training dataset, commonly referred to as Technologies within the project), the application automatically activates a second specialisation questionnaire.

This design decision is grounded in a pedagogical and usability rationale: the technology field is, within the scope of Lodz University of Technology (TUL), the broadest in terms of the number of available degrees, making a generic recommendation of "Technologies" insufficient to usefully guide the student. The second questionnaire therefore addresses specialisation within this field. The condition is evaluated in the main flow of app.py via a conditional structure that checks whether the value stored in st.session_state for the predicted field equals "Technologies", and if so calls the function responsible for rendering the second form in the interface.

7.3.2 Structure and Content of the Second Questionnaire

The second questionnaire is designed to refine the recommendation within the technology area by exploring the student's more specific preferences: affinity for programming or hardware, interest in industrial automation, biotechnology, architecture, civil engineering, or other specialisations offered by TUL. As with the first, its responses are collected via Streamlit widgets and validated before submission.

7.3.3 Prediction via the Third Model: Technological Specialisation

The responses from the second questionnaire, once validated, are encoded into a second numerical vector and passed to a third model — a second Random Forest classifier trained specifically to predict the most suitable degree programme or faculty within TUL's technology offering. This model was trained on a subset of the original dataset, filtered to include only the profiles of students classified under the technology field.

The result of this prediction is a more granular recommendation: a specific degree programme or a reduced set of related programmes, accompanied by a brief description and information about the available study programmes.

7.3.4 Displaying the Specialisation Result and Updating the Final JSON

The results of the second questionnaire and the third model are presented to the user in an integrated manner alongside those of the first questionnaire, forming a complete two-level recommendation: general field and technological specialisation. This information is also incorporated into the output JSON, updating the field corresponding to the specific prediction. If the user has requested to receive the results by email, the message body will likewise include the specialisation recommendation.

7.4 From Predictions to Visible Results: The User Experience

The design of Orient AI's presentation layer deliberately conceals the technical complexity of the underlying pipeline. The user interacts with a guided, conversational interface that presents each step sequentially: first the general questionnaire, then the results, and if necessary the specialisation questionnaire. At no point is the user exposed to technical terms such as "input vector", "Random Forest", or "linear regression"; instead, the results are framed in the language of academic guidance.

From a technical standpoint, this presentation is achieved through the use of Streamlit containers (st.container(), st.expander()), columns (st.columns()), and display elements such as st.metric(), st.info(), and st.success(). The transition between the "prediction in progress" state and the "results available" state is managed through boolean variables in st.session_state, which control which blocks of the interface are visible at each moment.

7.5 Conclusion: Orient AI as a Functional Prototype of AI-Based Academic Guidance

The integration described in this section makes Orient AI qualitatively different from a simple orientation questionnaire. A conventional questionnaire is limited to following predefined decision trees or assigning weighted scores to fixed categories, without any capacity for generalisation or learning from data. Orient AI, by contrast, grounds its recommendations in machine learning models trained on real student profile data, giving it the ability to capture complex, non-linear patterns between a student's characteristics and their most suitable academic field.

The chained pipeline that underpins the application — Random Forest for general field classification, linear regression for satisfaction estimation, and a second Random Forest for

technological specialisation — represents a layered inference architecture that progressively increases the precision of the recommendation. The serialisation of results into JSON, the email delivery of recommendations, and the conditional branch for the Technology area are indicators that the system does not merely produce correct predictions, but integrates them into a coherent and functional user flow.

In summary, Orient AI demonstrates that it is possible to build, within the context of a university project and using open-source tools, an intelligent academic guidance system that combines an accessible interface with real machine learning models, offering secondary school students a data-driven tool to support their vocational decision-making.